Pipelining

Shobhanjana Kalita,

Dept. of CSE, Tezpur University

Slides courtesy: *Computer Organization*, 6th Ed, C Hamacher et al, and *Computer Architecture and Organization*, 9th Ed, W. Stallings

Performance

- The speed at which a computer executes a program is affected by
  - the design of its hardware – processor speed, clock rate etc.
  - the machine language (ML) instructions – the instruction format, instruction set etc
    - the compiler that translates HLL programs into ML programs – how efficient the ML code generated by the compiler is.

Processor Clock

- Processor circuits are controlled by a timing signal called the **processor clock**
  - It defines a regular time intervals called clock cycles
- To execute a m/c instruction processor divides the actions to be taken into a sequence of basic steps
  - Each basic step can be executed in one **clock cycle**
- Length (P) of the clock cycle affects processor performance
  - Clock rate, $R = 1/P$
  - P is measured in cycles per second (Hertz, Hz)

Performance Equation

- Let T be the processor time required to execute a program written in a HLL
  - N is the number of machine language instructions generated corresponding to the program
  - S is the average basic number of steps needed to execute a single machine instructions
  - R is the clock rate
- The program execution time T is given as

$$T = \frac{N \times S}{R}$$

- This is called the **basic performance equation**

Performance Equation

- Performance of a computer is enhanced if the performance parameter T must be small
  - To reduce T: reduce N and S and increase R
- N, S and R are interdependent
  - The value of N is reduced if there are fewer machine instructions – *usually comes at the cost of greater number of basic steps per instruction*
  - The value of S is reduced if the average number of basic steps is reduced – *usually comes at the cost of greater number of instructions*
  - Increasing the value of R reduces the length of the clock cycle, *thereby giving less time for execution of a basic step*
- Need to introduce features that collectively brings down T

Pipelining

- Normally it is assumed that instructions are executed one after another, therefore
  - The total number of basic steps for a program

    $$S_1 + S_2 + \ldots + S_N = N \times S$$

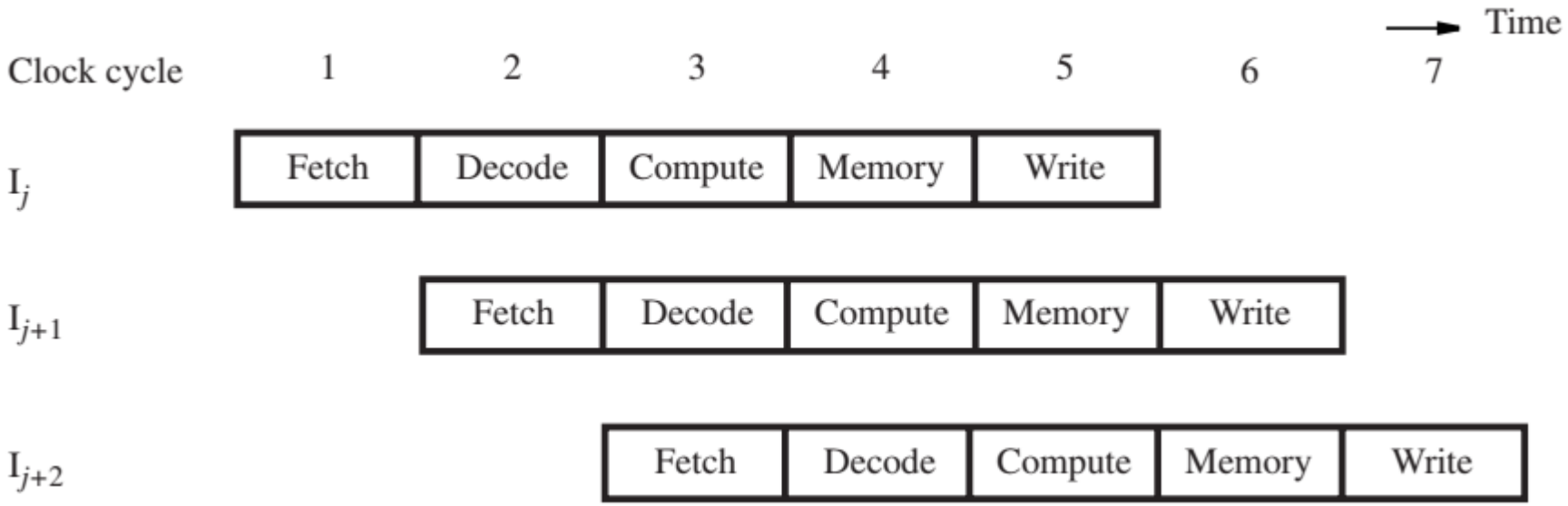
    where $S_1, S_2 \ldots S_N$ are the individual number of basic steps for each of the N instructions of a given program and S is the average number of basic steps
  - $N \times S$ determines the total number of basic steps required to execute program; this in turn determines the total number of clock cycles required to execute a program
- The total number of clock cycles to execute a program may be reduced by overlapping execution of the basic steps of successive instructions in the program – this is **pipelining**

Pipelining

- Pipelining takes advantage of the fact that the execution of any instruction can be broken down into a sequence of basic steps which are handled by different hardware units inside the processor
- For example, let us assume a 5-stage processor with the following stages:
  - Fetch: to fetch instruction from memory; performed by the fetch unit and system bus
  - Decode: to decode instruction and identify operand registers; performed by the decode unit in the control unit
  - Compute: to perform any computation; performed by the ALU
  - Memory: to read or write memory data if the instruction involves a memory operand using system bus
  - Write: to write memory data if the instruction involves a memory operand

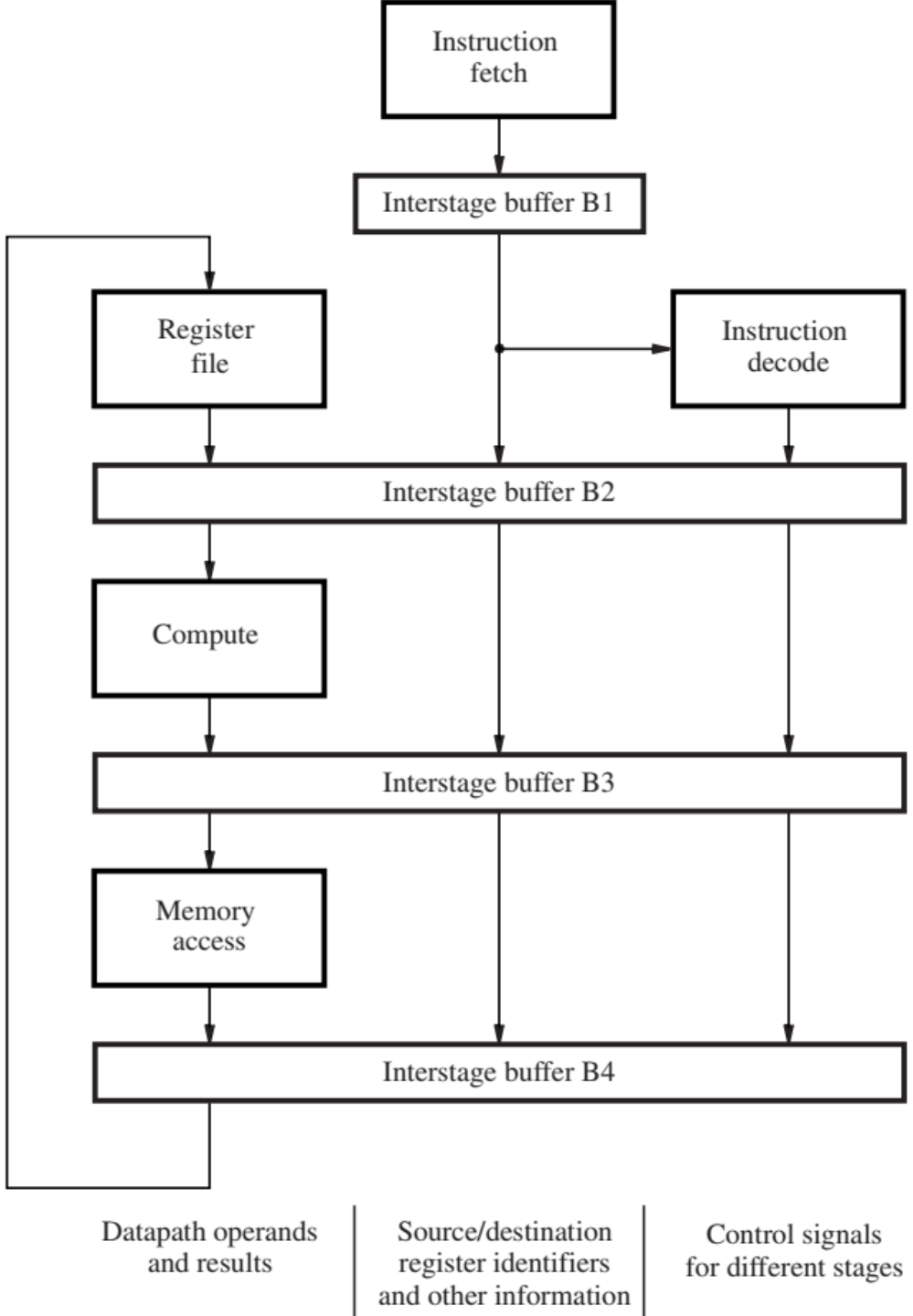
Pipelining

- In pipelining, different instructions at different stages of their execution are executed at the same time
  - It is assumed that each of the basic steps require the same amount of time i.e. 1 clock cycle
  - Completes the processing of one instruction in each cycle
  - Figure below shows a 5-stage processor



Pipeline Organization

- Buffer B1 feeds the **Decode** stage with a newly-fetched instruction.
- Buffer B2 feeds the **Compute** stage with the *operands* (with src/destn identifiers) from register file, *immediate value*, *incremented PC* value, and *control signals.* Control signals move through the pipeline to determine ALU operation, memory operation/ write operation to the register file.
- Buffer B3 holds the *result of ALU* operation, which may be written into register file or a *memory address*. It feeds the **Memory** stage; if memory r/w needed then it is performed.
- Buffer B4 feeds the **Write** stage with *values to be written into the register* file. This value may be the ALU result from the Compute stage, the result of the Memory access stage, or the incremented PC value that is used as the return address for a subroutine-call instruction.



Pipelining – Hazards

- Any condition that causes problems in the pipelined execution of a program is called a **hazard**
- Three types of hazards
  - **Structural Hazards** – when an instruction in the pipeline needs a hardware resource being used by another instruction
  - **Data Hazards** – when an instruction depends on a data value produced by an instruction still in the pipeline
  - **Control/Instruction Hazards** – when whether an instruction will be executed or not is determined by a control instruction which is still in the pipeline

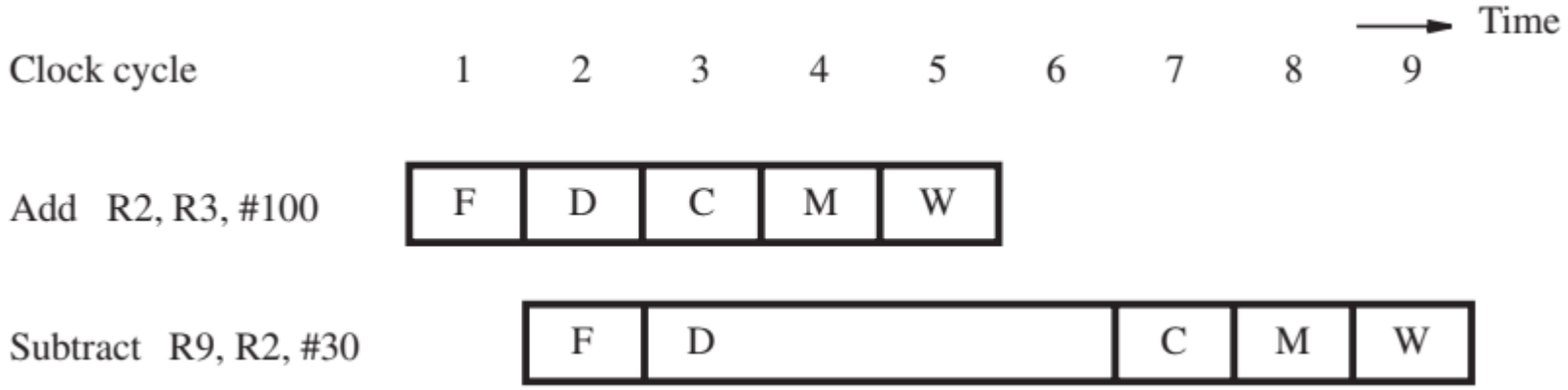
Structural Hazard

- Occurs when there are insufficient h/w resources to permit all actions to proceed concurrently.
- To deal with such situations:
  1. **Stalling** – postponing a step in the latter instruction to avoid collision
  2. **Add more h/w resources** to allow instructions to use independent resources at the same time

Clock cycle	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**
Instrn								
I1	F1	D2	C1	M1	W1			
I2		F2	D2	C2	M2	W2		
I3			F3	D3	C3	M3	W3	
I4				F4	D4	C4	M4	W4
I5						F5	D5	C5

Data Hazard

- Data hazard occurs when an instruction the pipeline is dependent on data that is modified by another instruction in the pipeline but has not yet been written
- For example:
  - `I1: Add R2, R3, #100` `[R3 + 100 -> R2 ]`
  - `I2: Sub R9, R2, #30` `[R2 – 30 -> R9]`
- Here the destination register of I1 is used as the source for I3. Unless I1 completes the write stage, I2 cannot complete the compute stage.

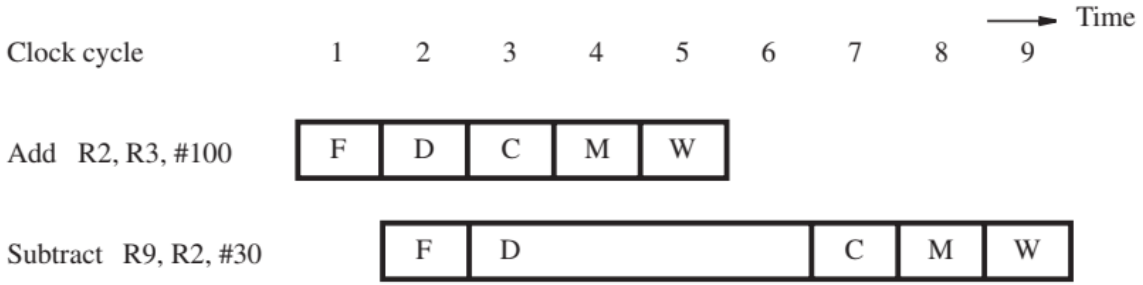


Data Hazard

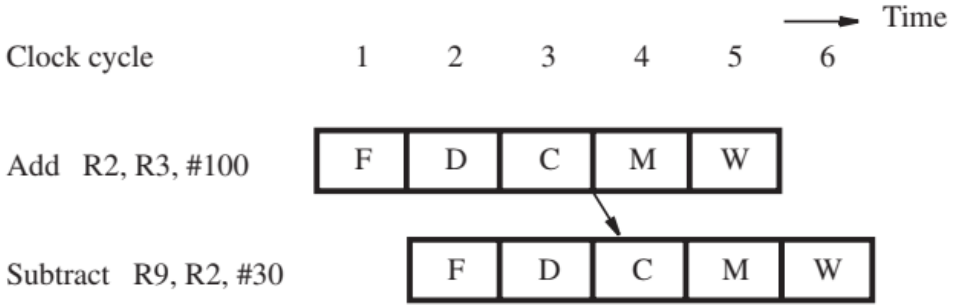
- Control unit must detect data hazard by comparing source register for I2 with the destination register for I1.
  - If it matches, then hold I2 in interstage buffer B1 while I1 proceeds
  - Control unit sends NOP (No Operation) signals to B2 instead of I2 – introducing bubbles (1 bubble = 1 clock cycle of idle time)
  - This manner of handling data hazard is **stalling**
- Ways in which data hazard can be handled are:
  - Stalling
  - Operand Forwarding
  - Software based handling

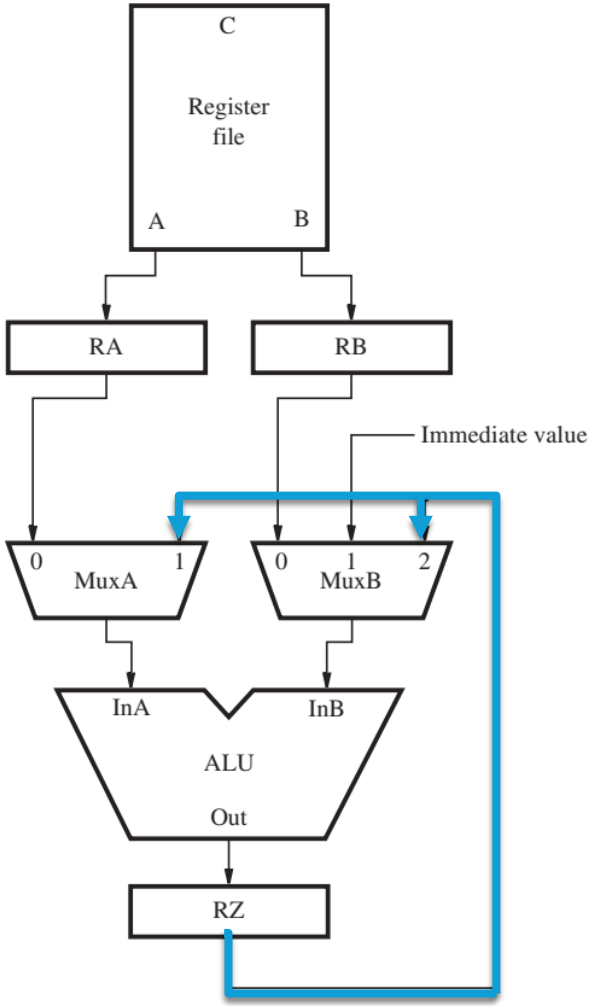
Data Hazard

Operand Forwarding



- In the example, pipeline had to be stalled till cycle 5, although result was available by end of cycle 3.
- If, instead, the hardware forwards the value from output of ALU to a multiplexer connected to the ALU input, then stalling may be avoided
- The multiplexers select a value from the register file (as done normally), or the value forwarded from the output ALU.



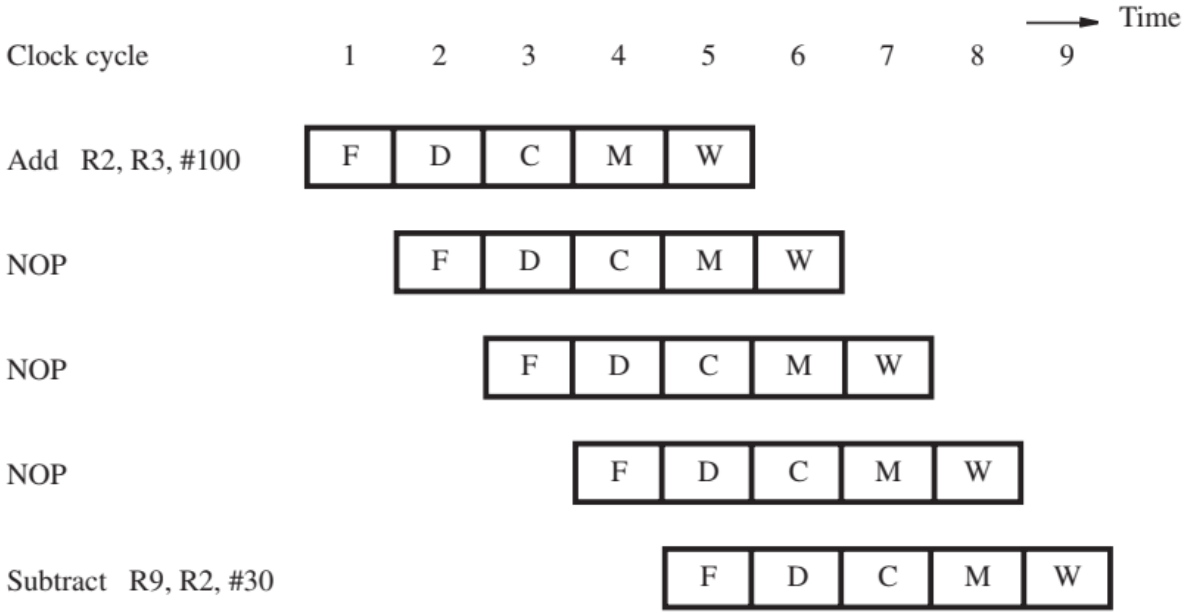


Data Hazard

Software based handling

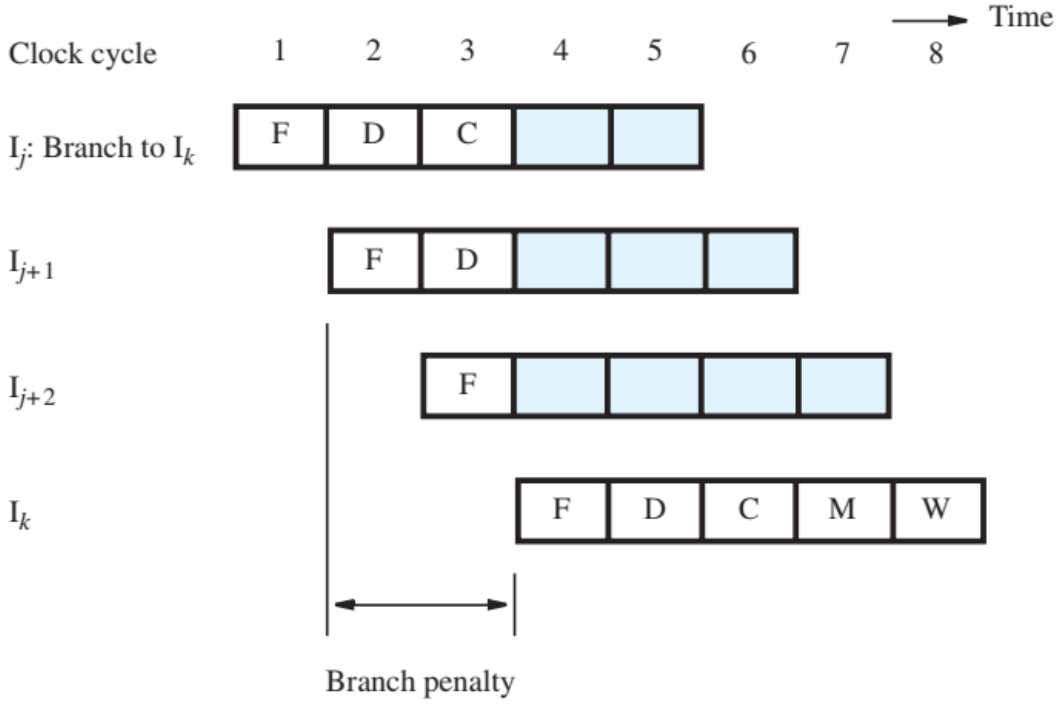
- The task of detecting and dealing with data dependencies can be left to the compiler.
- When compiler identifies data dependency between consecutive instructions, it introduces a NOP to delay the execution of the second instruction
- It has the same effect as stalling without introducing complexities in the h/w

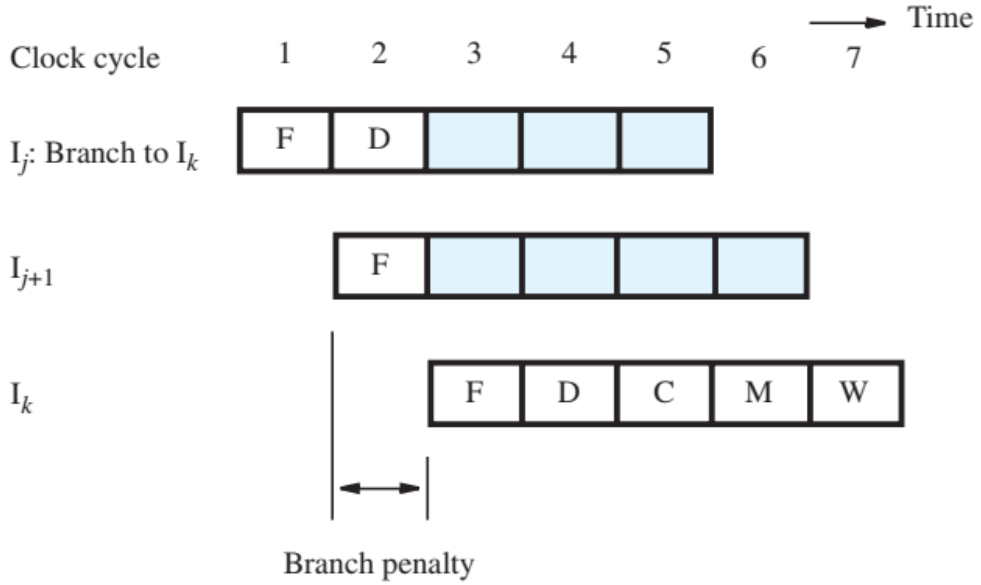
```
Add       R2, R3, #100
NOP
NOP
NOP
Subtract  R9, R2, #30
```



Control Hazard

- Control hazard occurs when conditional/ unconditional jump instruction is in the pipeline that requires jumping to a different location, but the immediate next instructions have already been fetched due to the pipelining
- If target address is determined at the compute stage, then a two-cycle delay is caused – *branch penalty*
- But if branch address is computed at the decode stage itself then penalty is reduced.





Control Hazard

Conditional Branch

- In case of conditional branch, the comparator that checks for conditions also need to be moved to the decode step
  - This reduces the branch penalty to one cycle but will be incurred at all branch instructions

Other ways of dealing with conditional branch

1. **Reorder the instruction** sequence -
   - The *branch delay slot* is used to execute instructions that do not determine the branch condition and will always be executed regardless of whether the branch is taken or not.

LOOP	Shift_left	R1
	Decrement	R2
	Branch=0	LOOP
NEXT	Add	R1,R3

(a) Original program loop

LOOP	Decrement	R2
	Branch=0	LOOP
	Shift_left	R1
NEXT	Add	R1,R3

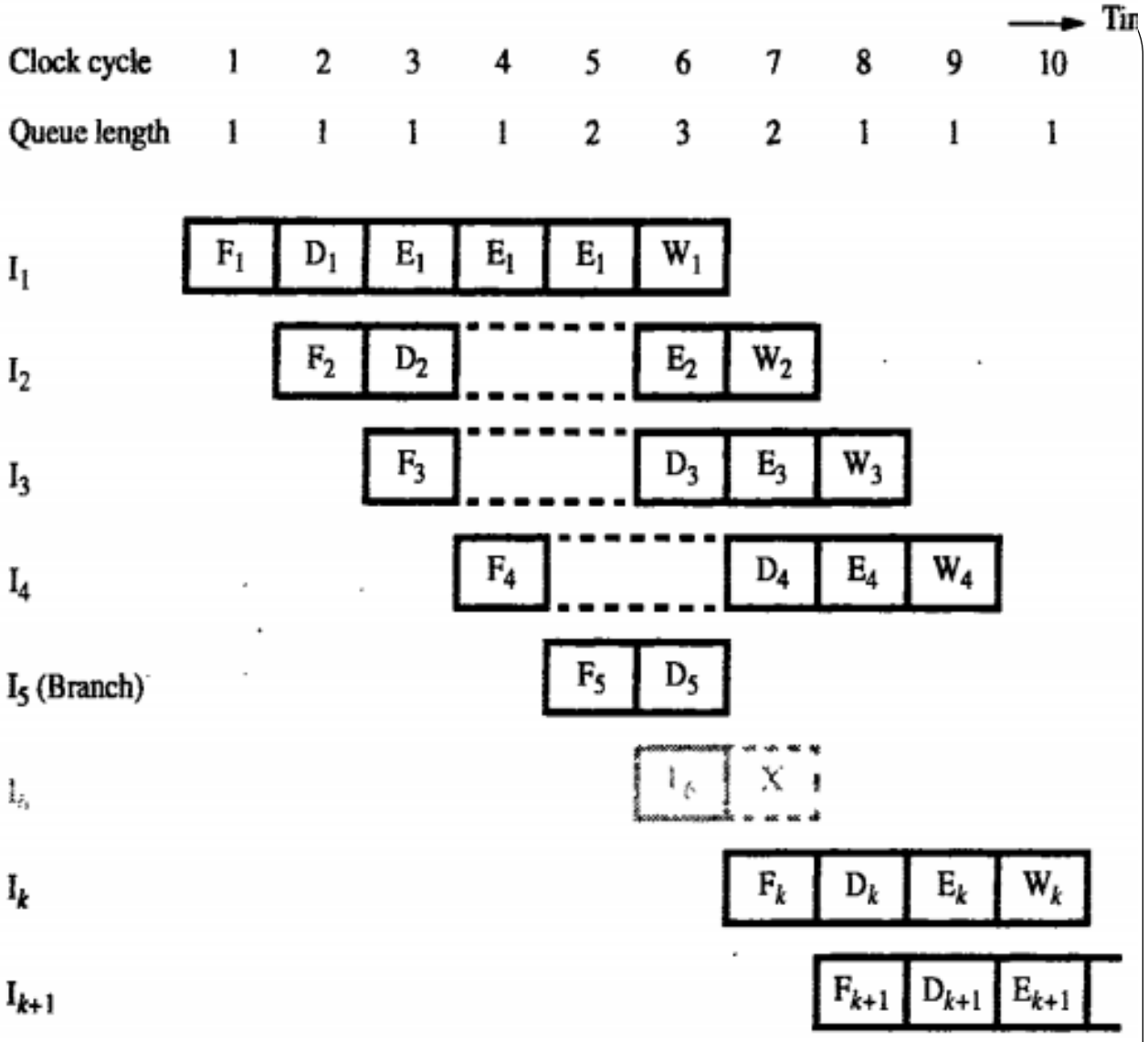
(b) Reordered instructions

Control Hazard

2. **Static Branch Prediction** – In case of conditional branching predict whether a branch will be taken or not and fetch/execute next instruction accordingly. The predicted instruction is executed but results are not written into.
   - Predict based on some heuristic such as –
     - Branch is always taken
     - Branch is taken 50% of the time
     - Take if branch instruction is in the beginning of a loop
3. **Dynamic Branch Prediction** – In static branch prediction the branch taken/not taken decision will be the same for all cases. At some point, the static prediction will lead to a wrong decision. In dynamic prediction, decision is made by looking at the instruction execution history
   - The probability of a branch being taken or not depends on the branch decisions taken so far.

Instruction Queuing and Prefetching

- When the pipeline stalls because of a data hazard, the fetch unit continues to fetch instructions to fill the instruction queue
- If there is a delay in fetching instructions due to a cache miss or branching delay, the dispatch/decode unit continues to work with the instructions in the queue
- To be effective the fetch unit must also have sufficient decoding and processing ability to recognize and execute branch instruction



To reduce *branch delay,* branch address is computed as soon as it is fetched

CISC/ RISC

- CISC – Complex Instruction Set Computer
  - Allows different instructions to have different number and sequence of basic steps
  - Allows instructions of different length
- RISC – Reduced Instruction Set Computer
  - All instructions have a fixed length of 1 word
  - All instructions require equal execution time

RISC

- Disadvantages of CISC
  - Complex compilers – complex machine instructions are often hard to exploit because compiler needs to find the exact machine instructions that fit the HLL construct. Having a simple (reduced) instruction set means that there are fewer instructions to choose from
  - Smaller programs are not necessarily fast – ML programs with fewer set of instructions ensures that the space taken by the program is less; but the number of basic steps to execute each instruction is more

RISC

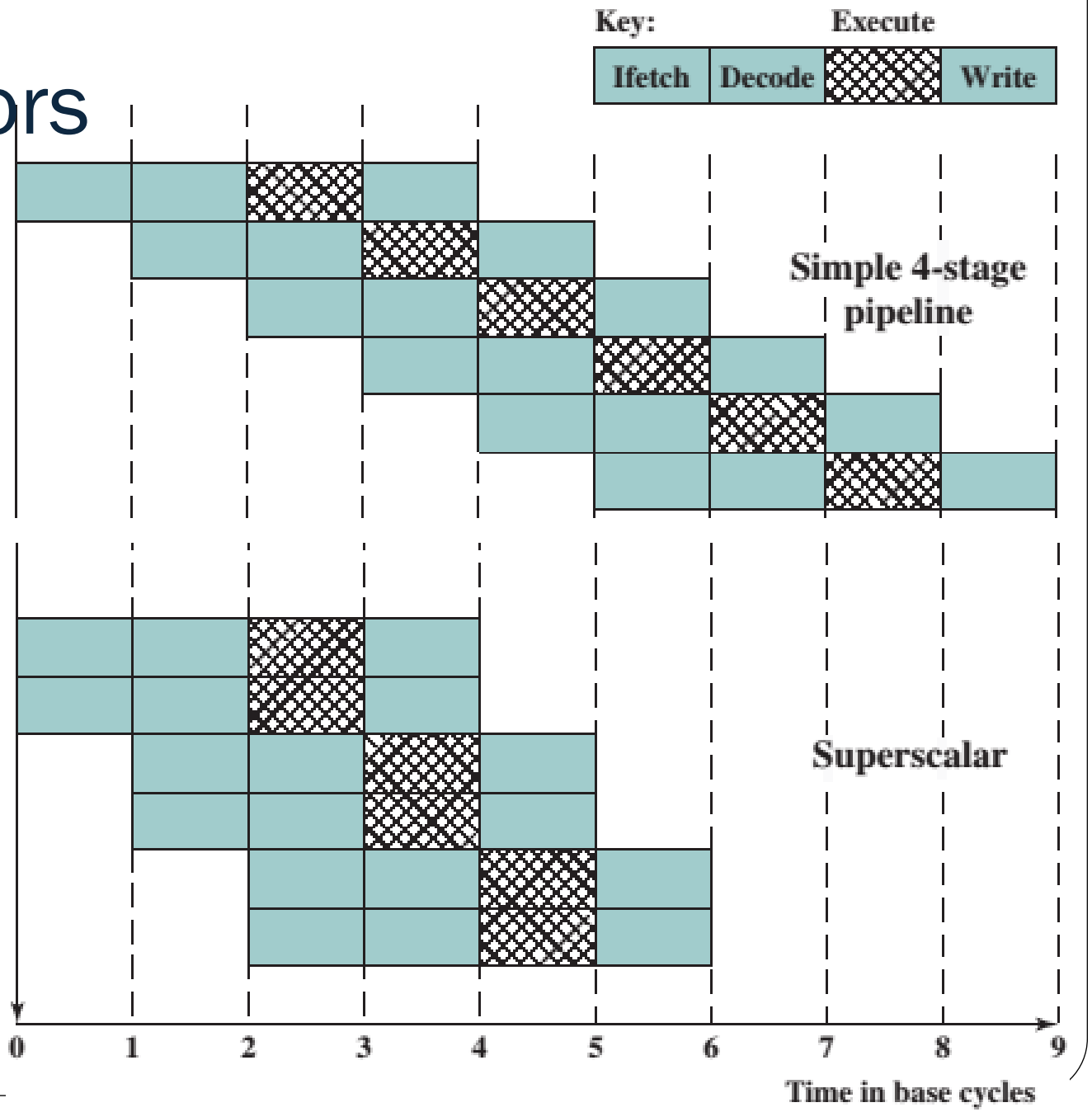
- Characteristics of RISC processors:
  - **One instruction per machine cycle –** With simple, one-cycle instructions, there is little or no need for microcode; the machine instructions can be hardwired. Such instructions should execute faster
  - **Register-to-register operations –** most operations are register to register, with only simple LOAD and STORE operations accessing memory. This design feature simplifies the instruction set and therefore the control unit
  - **Simple addressing modes –** Almost all RISC instructions use simple register addressing; simplifies the instruction set and the control unit
  - **Simple instruction formats –** Only one or a few formats are used. Instruction length is fixed; Field locations (especially opcode) are fixed. Benefits are:
    - Opcode decoding and register operand accessing can occur simultaneously.
    - Simplified formats simplify the control unit.
    - Instruction fetching is optimized because unit word-length unit are fetched.
- Pipelining is optimized easily due to these RISC features

Superscalar Processors

- The term *superscalar* refers to a machine that is designed to improve the performance of the execution of scalar instructions
  - ability to execute instructions independently and concurrently in different pipelines.
- In a traditional scalar organization, there is a single pipelined functional hardware unit
  - Parallelism is achieved by enabling multiple instructions to be at different stages of the pipeline at one time.
  - In the superscalar organization, there are multiple functional hardware units, each of which is implemented as a pipeline.

Superscalar Processors

- The superscalar approach depends on the ability to execute multiple instructions in parallel.
- **Instruction-level parallelism** refers to the degree to which, on average, the instructions of a program can be executed in parallel
- Data and control hazards are even more difficult to deal with



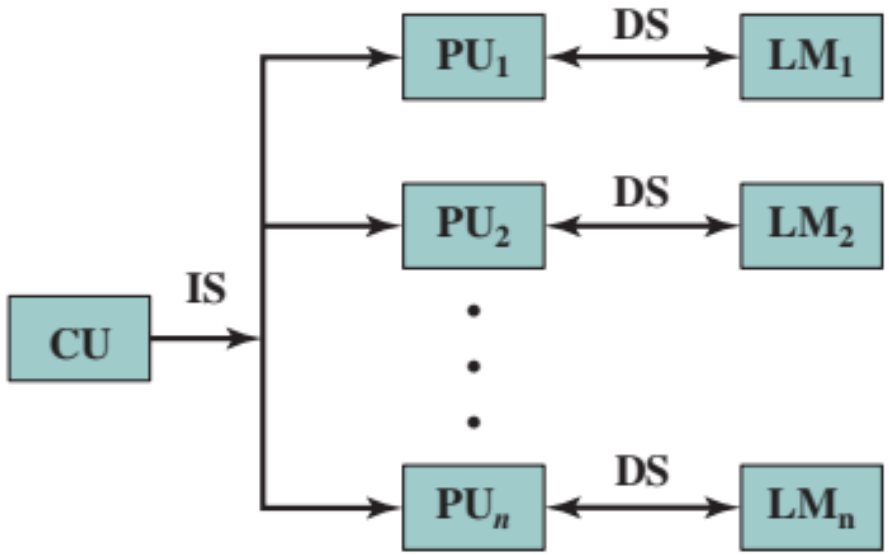
Flynn's Taxonomy

- A taxonomy of different types of parallel processors was put forward by Flynn, as follows,-
  - **Single Instruction Single Data (SISD) stream** - A single processor executes a single instruction stream to operate on data stored in a single memory. E.g. - Uniprocessors
  - **Single Instruction Multiple Data (SIMD) stream** - A single machine instruction controls the simultaneous execution of a number of processing elements on a lockstep basis. Each processing element has an associated data memory, so that instructions are executed on different sets of data by different processors.
  - **Multiple Instruction Single Data (MISD) stream -** A sequence of data is transmitted to a set of processors, each of which executes a different instruction sequence. Not available commercially
  - **Multiple Instruction Multiple Data (MIMD) stream** - A set of processors simultaneously execute different instruction sequences on different data sets

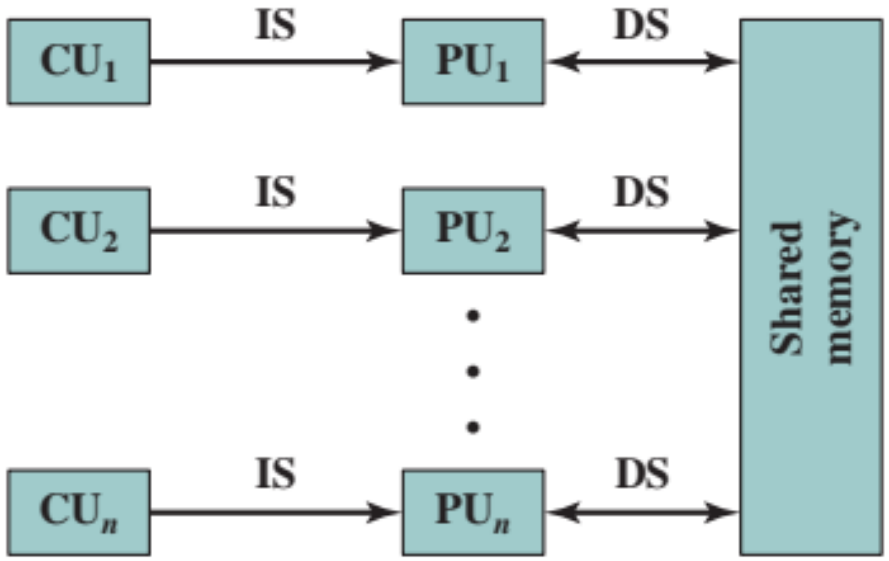
Parallel Processors

- MIMDs can be further subdivided by the means in which the processors communicate
  - **Multiprocessors:** Processors share a common memory; each processor accesses programs and data stored in the shared memory, and processors communicate with each other via that memory.
  - **Multicomputers:** Processors have individual memory areas; these are basically collection of independent uniprocessors/multiprocessors and communication among the computers is either via fixed paths or some network facility. Also called clusters.

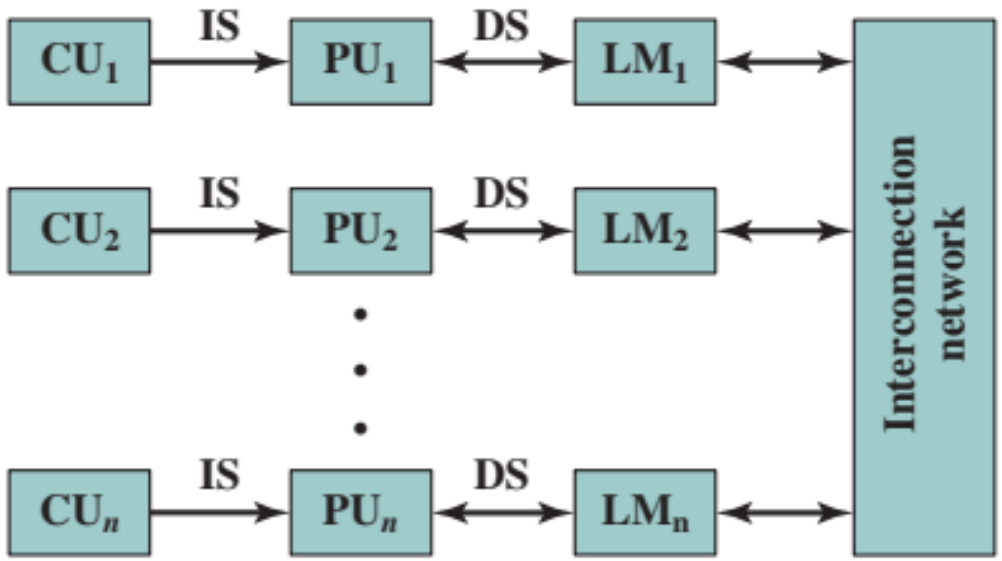
(a) SISD



(b) SIMD (with distributed memory)



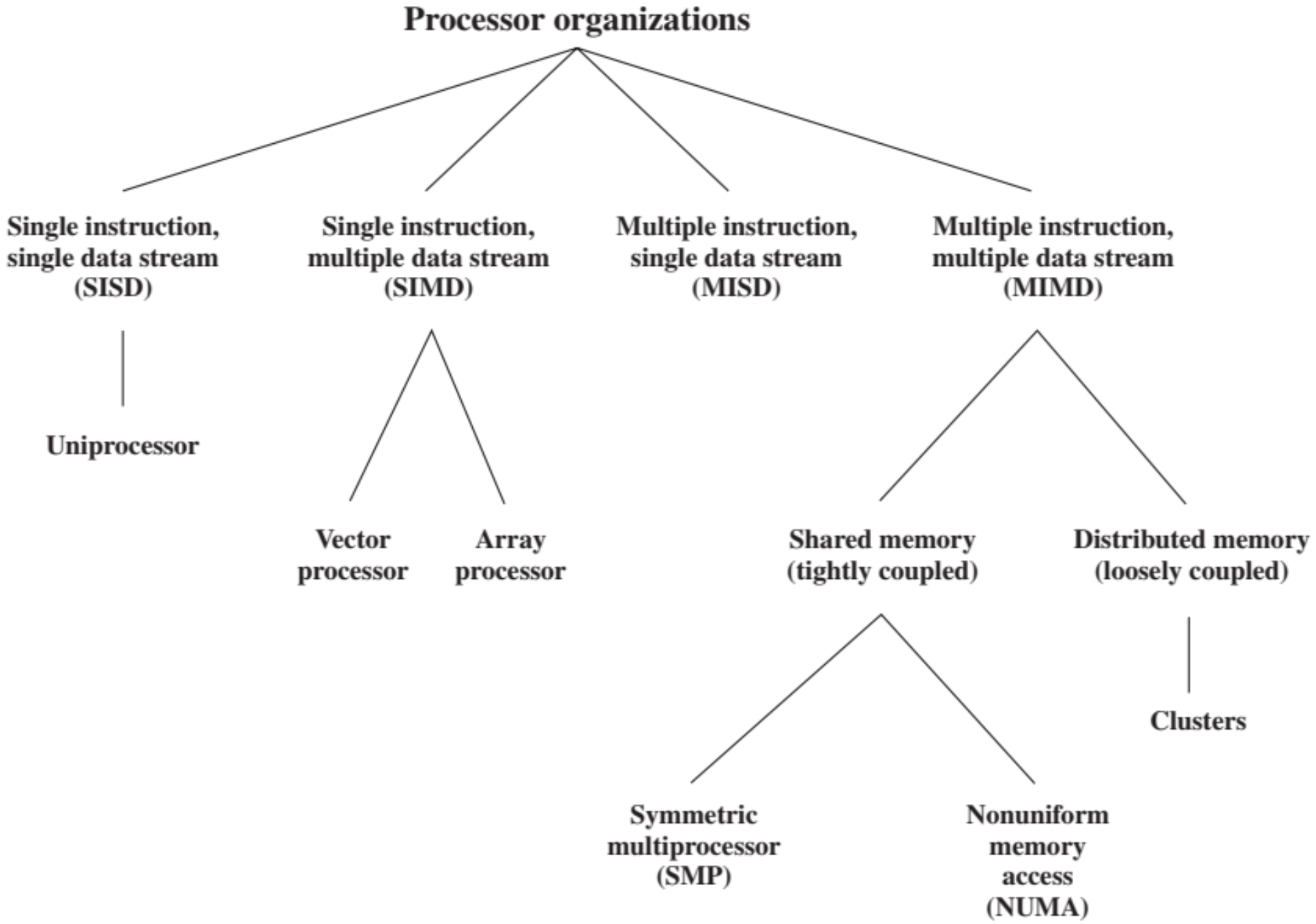
(c) MIMD (with shared memory)



(d) MIMD (with distributed memory)

CU = Control unit
IS = Instruction stream
PU = Processing unit
DS = Data stream
MU = Memory unit
LM = Local memory

SISD = Single instruction, single data stream
SIMD = Single instruction, multiple data stream
MIMD = Multiple instruction, multiple data stream


Processor organizations
Single instruction, single data stream (SISD)
Single instruction, multiple data stream (SIMD)
Multiple instruction, single data stream (MISD)
Multiple instruction, multiple data stream (MIMD)
Uniprocessor
Vector processor
Array processor
Shared memory (tightly coupled)
Distributed memory (loosely coupled)
Clusters
Symmetric multiprocessor (SMP)
Nonuniform memory access (NUMA)

Exercises

- Consider the following instructions at the given addresses in the memory:

1000 Add R3, R2, #20	[R2] + #20 → [R3]
1004 Subtract R4, R4, #3	[R4] − #3 → [R4]
1008 And R6, R4, #0x3A	[R4] & #0x3A → [R6]
1012 Add R7, R2, R4	[R2] + [R4] → [R7]

Initially, registers R2 and R4 contain 2000 and 50, respectively. These instructions are executed in a computer that has a five-stage pipeline with stages Fetch, Decode, Compute, Memory, Write. The first instruction is fetched in clock cycle 1, and the remaining instructions are fetched in successive cycles.

(*a*) Draw a diagram that represents the flow of the instructions through the pipeline. Describe the operation being performed by each pipeline stage during clock cycles 1 through 8.

(*b*) Describe the contents of registers IR, PC, R1 to R7 in the pipeline during cycles 2 to 8.